



INTRODUCTION



Welcome to the second edition of *Python Playground*! Within these pages, you'll find 15 exciting projects designed to encourage you to explore the world of programming with Python. The projects cover a wide range of topics, such as drawing Spirograph-like patterns, 3D rendering, projecting laser patterns in sync with music, and speech recognition using machine learning. In addition to being fun in and of themselves, these projects offer a lot of room for expansion and are designed to be jumping-off points for you to explore your own ideas.

Who Is This Book For?

Python Playground was written for anyone curious about how to use programming to understand and explore ideas. The projects in this book assume that you know basic Python syntax and basic programming concepts and that you're familiar with high school mathematics. I've done my best to explain in detail the math you need for all projects.

This book isn't intended to be your first book on Python. I won't walk you through the basics. I will, however, show you how to use Python to solve a variety of real-world problems in a series of nontrivial projects. As you work through the projects, you'll explore the nuances

of the Python programming language and learn how to work with some popular Python libraries. But perhaps even more important, you'll learn how to break down a problem into parts, develop an algorithm to solve that problem, and then implement a solution from the ground up using Python.

It can be difficult to solve real-world problems because they are often open-ended and require expertise in various domains. But Python offers the tools to facilitate problem-solving. Overcoming difficulties and finding solutions to real problems are the most important parts of your journey on the way to becoming an expert programmer.

What's in This Book?

Let's take a quick tour through the chapters in this book. **Part I** features a few projects to get you started.

Chapter 1: The Koch Snowflake Uses recursive functions and `turtle` graphics to draw an interesting fractal shape

Chapter 2: Spirographs Harnesses parametric equations and `turtle` graphics to draw curves like the ones generated with the toy Spirograph

Part II features projects that use mathematical models to create computer simulations of real-world phenomena.

Chapter 3: Conway's Game of Life Implements a famous cellular automaton using `numpy` and `matplotlib`, producing an evolving, life-like system based on a few simple rules

Chapter 4: Musical Overtones with Karplus-Strong Realistically simulates the sound of a plucked string instrument and plays back the sounds with `pyaudio`

Chapter 5: Flocking Boids Uses `numpy` and `matplotlib` to implement the Boids algorithm and simulate the flocking behavior of birds

The projects in **Part III** will introduce you to reading and manipulating 2D images with Python.

Chapter 6: ASCII Art Introduces `Pillow`, a fork of the Python Imaging Library (PIL), by converting images into text-based ASCII art

Chapter 7: Photomosaics Stitches together a grid of smaller images to create a recognizable larger image

Chapter 8: Autostereograms Uses depth maps and pixel manipulation to embed the illusion of a 3D image in a 2D image

In **Part IV**, you'll learn to use shaders and the OpenGL library to quickly and efficiently render 3D graphics by working directly with your graphics processing unit (GPU).

Chapter 9: Understanding OpenGL Introduces the basics of using OpenGL to create simple 3D graphics

Chapter 10: Conway's Game of Life on a Torus Revisits the simulation from **Chapter 3** and brings it to life on the surface of a 3D torus

Chapter 11: Volume Rendering Implements a volume ray casting algorithm to render volumetric data, a technique commonly used for medical imaging such as MRI and CT scans

Finally, in **Part V**, you'll work with a Raspberry Pi and other electronic components to deploy Python on embedded systems.

Chapter 12: Karplus-Strong on a Raspberry Pi Pico Creates a playable electronic instrument by implementing the Karplus-Strong algorithm from **Chapter 4** on a tiny Raspberry Pi Pico microcontroller using MicroPython

Chapter 13: Laser Audio Display with a Raspberry Pi Harnesses Python on a Raspberry Pi to control two rotating mirrors and a laser, producing a laser light show that responds to sound

Chapter 14: IoT Garden Uses Bluetooth Low Energy to connect a Raspberry Pi with Adafruit hardware running CircuitPython, creating an IoT system to monitor temperature and humidity levels in your garden

Chapter 15: Audio ML on Pi Introduces the exciting world of machine learning with TensorFlow by implementing a speech recognition system on your Raspberry Pi

Each chapter ends with an “Experiments!” section featuring suggestions for how you can expand the project or further explore the topic at hand.

What’s New to the Second Edition?

This second edition features five new projects, including the Koch snowflake project in **Chapter 1** and the 3D version of Conway’s Game of Life in **Chapter 10**. The most significant updates are in the hardware section, which has been streamlined to focus exclusively on Raspberry Pi–based systems, rather than a mix of Raspberry Pi and Arduino. As such, every project in **Part V** either is new (**Chapter 12**, **Chapter 14**, **Chapter 15**) or has been thoroughly redesigned (**Chapter 13**).

Relying just on Raspberry Pi simplifies the setup process for the hardware projects, and it keeps the book’s focus on Python programming. There’s no longer a need to switch between Python and code written in the Arduino programming language (a version of C++). With the updated **Part V**, you’ll also get a taste for programming with MicroPython and CircuitPython, two flavors of Python optimized for running on resource-constrained embedded systems.

Other significant updates to the second edition include:

- Using `pyaudio` rather than `pygame` to play WAV files in [Chapter 4](#)
- Comparing the performance of a linear search algorithm and a k-d tree data structure when finding the best image matches for the photomosaic in [Chapter 7](#)
- Guidance on how to create your own depth maps for generating autostereograms in [Chapter 8](#)
- Streamlined installation instructions in [Appendix A](#) using the standard Anaconda Distribution of Python

Beyond these specific updates, the entire text has been reviewed, corrected, and clarified, and the code has been updated as needed to reflect changes to Python since the first edition was released.

Why Python?

Python is an ideal language for exploring programming. As a multi-paradigm language, it provides you with a lot of flexibility in how you structure your programs. You can use Python as a *scripting* language to simply execute code, as a *procedural* language to organize your program into a collection of functions that call one another, or as an *object-oriented* language that uses classes, inheritance, and modules to build up a hierarchy. This flexibility allows you to choose the programming style most suited to a particular project.

When you develop using a more traditional language like C or C++, you have to compile and link your code before you can run it. With Python, you can run it directly after editing. (Under the hood, Python compiles your code into an intermediate bytecode that is then run by the Python interpreter, but these processes are transparent to you, the user.) In practice, the process of modifying and running your code over and over is much less cumbersome with Python.

Python has a very small set of simple and powerful data structures. If you already understand strings, lists, tuples, dictionaries, list comprehensions, and basic control structures such as `for` and `while` loops, you're off to a great start. Python's succinct and expressive syntax makes it easy to do complex operations with just a few lines of code. And once you're familiar with Python's built-in and third-party modules, you'll have an arsenal of tools to tackle real problems like the ones covered in this book. There are standard ways to call C/C++ code from Python and vice versa, and because you can find libraries to do almost anything in Python, it's easy to combine Python with other language modules in bigger projects. This is why Python is considered a great *glue* language—it makes it easy to combine diverse software components. The 3D graphics projects in [Part IV](#) demonstrate how Python can work side by side with the C-like OpenGL Shading Language, and in [Chapter 14](#) you'll even mix in a little bit of HTML, CSS, and JavaScript to create a web interface for your IoT garden monitor. Real software projects often use a mix of software technologies, and Python fits very well into such layered architectures.

Python also provides a handy tool in the form of the Python interpreter. It lets you easily check code syntax, do quick computations, and even test code under development. When I write Python code, I keep three windows open: a text editor, a shell, and a Python interpreter. As I develop code in the editor, I import my functions or classes into the interpreter and test them as I go.

I also use the interpreter to get a feel for new modules before I incorporate them into my code. For example, while developing code for the IoT garden project in [Chapter 14](#), I wanted to test the `sqlite3` database module. I brought up the Python interpreter and tried the following to make sure I understood how to create and add entries to a database:

```
>>> import sqlite3
```

```
>>> con = sqlite3.connect('test.db')

>>> cur = con.cursor()

>>> cur.execute("CREATE TABLE sensor_data (TS datetime, ID
text, VAL numeric)")

>>> for i in range(10):

...   cur.execute("INSERT into sensor_data VALUES
(datetime('now'),'ABC', ?)", (i, ))

>>> con.commit()

>>> con.close()

>>> exit()
```

Then, to make sure it worked, I did the following to retrieve some of the data I added:

```
>>> con = sqlite3.connect('test.db')

>>> cur = con.cursor()

>>> cur.execute("SELECT * FROM sensor_data WHERE VAL > 5")

>>> print(cur.fetchall())
```

```
[('2021-10-16 13:01:22', 'ABC', 6), ('2021-10-16 13:01:22', 'ABC', 7),
 ('2021-10-16 13:01:22', 'ABC', 8), ('2021-10-16 13:01:22', 'ABC', 9)]
```

This example demonstrates the practical use of the Python interpreter as a powerful tool for development. You don't need to write a complete program to do quick experiments; just open up the interpreter

and get going. This is just one of the many reasons why I love Python and why I think you will too.

The Code in This Book

I've done my best throughout this book to walk you through the code for each project in detail, piece by piece. You can either enter the code yourself or download the complete source code (using the Download Zip option) for all programs in the book at

<https://github.com/mkvenkit/pp2e>.

You'll find several exciting projects in the following pages. I hope you have as much fun playing with them as I had creating them. Don't forget to explore further with the experiments presented at the end of each project. I wish you many happy hours of programming with *Python Playground*!